Model Compression

Dhruv Kumar

BITS Pilani

Pruning: remove weights entirely

Pruning sets a fraction of a model's weights to exactly zero, based on some importance criterion — the simplest version compares magnitude to a threshold:

$$w_i \leftarrow \begin{cases} 0 & |w_i| < \tau \\ w_i & \text{otherwise} \end{cases}$$

w_i — one weight; τ — a threshold chosen so that a target fraction of all weights (the **sparsity**) falls below it and gets zeroed.

> WHY SPARSE NETWORKS CAN STILL WORK
>
> The **Lottery Ticket Hypothesis** showed that dense, randomly-initialized networks contain sparse "winning ticket" subnetworks that, trained in isolation from their original initialization, match the full network's accuracy — often at under 10–20% of the original parameters.

Pruning at LLM scale

Retraining a multi-billion-parameter model after pruning is itself expensive — these two methods prune in **one shot**, from a small calibration set, with no retraining.

SPARSEGPT

For each layer, finds a sparse weight matrix $\hat{W}$ minimizing the reconstruction error $\|WX - \hat{W}X\|^2$ on calibration data X, solved efficiently via second-order (Hessian-based) information. Prunes 175B-parameter models to 50–60% sparsity in under 5 hours, with little increase in perplexity.

WANDA

Scores each weight by magnitude *times* input activation size — no retraining or gradient updates needed:

$$S_{ij} = |W_{ij}| \cdot \|X_j\|_2$$

W_{ij} — one weight; X_j — the j-th input feature's activation norm over the calibration set; lowest-scoring weights per output are pruned.

Quantization: fewer bits per weight

Quantization maps a real-valued weight to a low-bit-width integer, using a scale and a zero-point:

$$x_{int} = \text{clamp}\left(\lfloor x/s \rceil + z,\ 0,\ 2^b - 1\right)$$

$$\hat{x} = s\,(x_{int} - z)$$

x — the original weight; s — the scale; z — the integer zero-point; b — the bit-width (e.g. $b = 4$ or 8); x_{int} — the stored integer; $\hat{x}$ — the dequantized approximation of x, used whenever the weight is actually multiplied.

Quantization at LLM scale

Naive uniform quantization breaks down on LLMs because a small number of **outlier** weights or activations dominate the range. These methods handle that:

LLM.int8()

Keeps almost all values in 8-bit, but routes the rare outlier feature dimensions to a 16-bit matrix multiply instead of quantizing them.

GPTQ

One-shot weight-only quantization to 3–4 bits, using Hessian-based layer-wise reconstruction — the same idea as SparseGPT, applied to quantization.

AWQ

Finds the ~1% of weight channels that matter most (by activation magnitude, not weight magnitude) and protects them with a per-channel scaling transform.

Knowledge distillation

A small **student** model is trained to match a large **teacher** model's output distribution, not just the hard labels.

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

z_i — the teacher's (or student's) logits; T — temperature: $T = 1$ is standard softmax, $T > 1$ softens the distribution so the student also learns from the teacher's runner-up predictions, not just its top choice.

The student's loss is a weighted sum of two cross-entropies — a **soft-target** loss against the teacher's softened distribution, and a **hard-target** loss against the true label — with the soft-target term scaled by T^2 to keep its gradient magnitude comparable to the hard-target term's.

> IN PRACTICE: DISTILBERT
>
> Distilling BERT this way gives a student that is **40% smaller**, runs **60% faster**, and retains **97%** of BERT's language-understanding performance.

Reading list

PRUNING

Frankle & Carbin, "The Lottery Ticket Hypothesis" (2019)
Frantar & Alistarh, "SparseGPT" (2023)
Sun et al., "Wanda" (2023)

QUANTIZATION

Dettmers et al., "LLM.int8()" (2022)
Frantar et al., "GPTQ" (2022)
Lin et al., "AWQ" (2023)

DISTILLATION

Hinton, Vinyals & Dean, "Distilling the Knowledge in a Neural Network" (2015)
Sanh et al., "DistilBERT" (2019)

FORMAL REFERENCE

Nagel et al., "A White Paper on Neural Network Quantization" (2021) — the general affine-quantization formulas used on this deck.